Project RAG

Project Introduction

My project is **"AUTOSAR HLD Intelligence & Traceability Assistant – Retrieval-Augmented Generation (RAG) System."** The objective is to provide accurate, grounded architectural answers and component traceability from uploaded automotive High-Level Design (HLD) PDF documents using RAG. The system uses RAG to retrieve relevant specification passages from the uploaded HLD document and generate accurate, verified answers with source citations. We developed this to overcome the limitation of normal generic LLMs, which give incorrect, hallucinated, or ungrounded answers when technical automotive information is not present in their training data or when proprietary confidential documents are analyzed.

2. Problem Statement

Interviewer: What problem does your project solve?

Automotive HLD documents follow strict AUTOSAR standards and span hundreds of pages detailing Software Components (SWCs), ports, interfaces, CAN signals, and Basic Software (BSW) stacks. Traditional Large Language Models depend only on their pre-trained knowledge. They cannot know proprietary information from newly uploaded automotive documents, and they frequently hallucinate signal names, port identifiers, or fake page references. Our project solves this by allowing automotive engineers to upload their own HLD PDFs and get verified, evidence-based answers directly from the document content with exact source-page citations.

What is your contribution in the project?

My contribution was mainly in implementing the end-to-end RAG pipeline, document preprocessing with page-level provenance, vector database creation using FAISS, two-stage Cross-Encoder re-ranking, and response verification with reasoning-trace stripping.

Document Preprocessing

- Extracting text page-by-page from automotive PDF files using PyMuPDF (fitz).
- Removing excessive whitespace and validating extractable text coverage.
- Splitting multi-page automotive specifications into smaller semantic chunks.
- Creating overlapping chunks while preserving document name and page number metadata.

Why is this needed?

An automotive HLD PDF may contain hundreds of pages with dense tables and architecture diagrams.
The LLM cannot directly search through the entire document efficiently due to context-window limits.
So we divide the document into smaller meaningful sections (chunks) before encoding and indexing them.

2. Document Chunking

We divide the document text into smaller pieces called **chunks**.

Why?

"LLMs cannot efficiently process very large documents at once, so we split the text into smaller meaningful sections with provenance metadata."

Project values:

- **Chunk Size = 800 characters** (captures full AUTOSAR signal & port interaction blocks)
- **Overlap = 150 characters**

The overlap ensures that critical automotive context (such as port connections split across lines) is not lost between consecutive chunks.

3. Embedding Generation

We convert every text chunk into dense numerical vectors called embeddings using the **sentence-transformers/all-MiniLM-L6-v2** embedding model.

Why?

"Computers and search indexes cannot understand text directly, so embeddings represent the semantic meaning of technical text numerically in a 384-dimensional vector space."

4. Vector Database Creation — FAISS

FAISS Vector Database

"All normalized chunk embeddings are indexed in a **FAISS (Facebook AI Similarity Search)** vector database."

Why?

"When an engineer asks an architecture question, FAISS quickly finds the most similar chunks using inner product cosine similarity in milliseconds without searching the entire document linearly."

5. Re-Ranking Optimization — Cross-Encoder

I implemented two-stage retrieval using a **Cross-Encoder re-ranker (cross-encoder/ms-marco-MiniLM-L-6-v2)** to re-score candidate chunks instead of relying solely on bi-encoder vector similarity.

Parameters Used:

- Initial FAISS Candidate Pool: **k = 15 chunks**
- Re-ranked Top Evidence Chunks: **top_k = 5 chunks**
- Similarity Metric: **Cosine Inner Product (IndexFlatIP)**

Why Cross-Encoder?

FAISS uses bi-encoders which compute embeddings independently. The Cross-Encoder scores query and passage simultaneously through full cross-attention layers, eliminating false-positive lexical matches and boosting technical retrieval precision.

6. Prompt Engineering & Output Structure

I designed structured system prompts that instruct the LLM to answer strictly from the retrieved document evidence and format answers into three clean sections:

- **Executive Summary** — High-level direct explanation of the architectural query.
- **Architectural Breakdown** — Components, Interfaces & Ports, Signals, and Functional Flows.
- **Evidence & Citations Summary** — Grounded table linking facts to document filenames and page numbers.

Why?

LLMs sometimes use their own pre-trained knowledge and hallucinate automotive facts. Strict prompt framing and programmatic post-verification eliminate hallucinations.

7. Reasoning Trace Stripping & Citation Verification

Local reasoning models (such as DeepSeek-R1) output internal chain-of-thought traces inside <*think*> ... </*think*> blocks. I built a post-processing cleaner that completely strips internal reasoning from the frontend display while programmatically validating document names and page numbers against retrieved chunks.

Technologies Used

What technologies did you use?

- **Python 3.11** — Core programming language.
- **Streamlit** — Web dashboard framework providing interactive multi-tab engineering UI.
- **PyMuPDF (fitz)** — Fast, accurate page-by-page PDF text and metadata extraction.
- **Sentence Transformers** — Used for generating dense text embeddings (all-MiniLM-L6-v2).
- **FAISS CPU** — Vector database for sub-millisecond semantic similarity search.
- **Cross-Encoder** — Deep neural re-ranking model (ms-marco-MiniLM-L-6-v2) for evidence precision.
- **Ollama (Local LLM)** — Provides 100% private, offline inference using deepseek-r1:1.5b.
- **OpenRouter / OpenAI API** — Optional cloud LLM integration for high-throughput synthesis.
- **pandas** — Structured architecture findings extraction and CSV export.
- **pytest** — Comprehensive automated test suite (78 tests passing).

Complete Workflow

PDF Upload (Streamlit UI)

↓

Text Extraction (PyMuPDF — page-by-page metadata preservation)

↓

Chunking (800 chars, 150 overlap, chunk ID & page coordinates)

↓

Embeddings (Sentence Transformers all-MiniLM-L6-v2 — 384 dimensions)

↓

FAISS Indexing (IndexFlatIP — persistent cosine vector index)

↓

User Question (Natural language architectural query)

↓

FAISS Candidate Recall (Top-15 most similar chunks retrieved)

↓

Cross-Encoder Re-Ranking (Top-5 high-relevance evidence passages selected)

↓

LLM Generation (Ollama deepseek-r1:1.5b / Extractive synthesis mode)

↓

Post-Processing (Strip reasoning traces + verify ground-truth page citations)

↓

Final Verified Output (Executive Summary + Architectural Breakdown + Citations Table)

What are Embeddings?

Embeddings are dense numerical vector representations of text that capture semantic meaning in high-dimensional vector space.

Why deepseek-r1:1.5b / Ollama?

Ollama with deepseek-r1:1.5b provides:

- **100% Local & Private:** Confidential automotive HLD architectures never leave the laptop.
- **High Reasoning Capability:** Strong technical reasoning for cross-module dependencies and port tracing.
- **Zero API Costs:** Runs efficiently on standard consumer CPU laptops without cloud tokens.

How did you evaluate the system?

We evaluated the system using:

- **Retrieval Relevance:** Proportion of domain-expected terms found in retrieved candidate passages.
- **Citation Correctness:** Programmatic verification ensuring 100% of cited page numbers match actual source text.
- **Automated Test Suite:** 78 automated unit and integration tests across all modules via pytest (100% pass rate).
- **10-Question Benchmark (evaluate.py):** Evaluated component extraction, interface mapping, CAN signals, and unsupported query handling.

Challenges Faced

What challenges did you face?

- Multi-hundred-page technical PDFs with complex formatting and tables.
- Vector similarity false positives where lexical matches lacked true architectural relevance (solved via Cross-Encoder).
- Small LLM models outputting reasoning monologues (<think>) and placeholder citations (solved via programmatic post-verification).
- Ensuring 100% functionality without paid APIs (solved via offline extractive mode).

Future Scope

What is the future scope?

- Hybrid Search combining dense semantic embeddings with sparse BM25 keyword matching.
- Native OCR ingestion for scanned legacy automotive engineering blueprints.
- ARXML (AUTOSAR XML) direct parsing alongside PDF specifications.

CODES SYNTAX

1. Load & Extract PDF (PyMuPDF)

</> Python — PDF Extraction

```
import fitz # PyMuPDF
doc = fitz.open(stream=pdf_bytes, filetype='pdf')
for page_num in range(len(doc)):
page = doc[page_num]
text = page.get_text('text')
pages.append(PageRecord(page_number=page_num + 1, text=text))
```

Interview Answer: *"I used PyMuPDF (fitz) to extract text page-by-page while preserving exact document names and page numbers for source citations."*

2. Document Chunking

</> Python — Chunking with Overlap

```
chunk_size = 800
overlap = 150
start = 0
while start < len(page_text):
end = min(start + chunk_size, len(page_text))
chunk = TextChunk(chunk_id=f'{doc_name}_p{page}_c{idx}', text=page_text[start:end])
chunks.append(chunk)
start += (chunk_size - overlap)
```

Interview Answer: *"I split large automotive HLD documents into 800-character chunks with 150-character overlap to preserve cross-sentence architectural context."*

3. Generate Dense Embeddings

</> Python — Sentence Transformers Embeddings

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(texts, normalize_embeddings=True)
# Generates 384-dimensional unit-normalized dense vectors
```

Interview Answer: *"I converted text chunks into 384-dimensional dense vectors using all-MiniLM-L6-v2 with L2-normalization for cosine distance."*

4. Create FAISS Vector Index

</> Python — FAISS Vector Index

```
import faiss
dim = embeddings.shape[1] # 384
index = faiss.IndexFlatIP(dim) # Inner Product similarity
index.add(embeddings)
scores, indices = index.search(query_vector, top_k=15)
```

Interview Answer: *"I stored the embeddings in a FAISS IndexFlatIP vector database to perform sub-millisecond cosine similarity search over candidate chunks."*

5. Cross-Encoder Re-Ranking

</> Python — Cross-Encoder Re-Ranking

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
pairs = [[query, chunk['text']] for chunk in candidate_chunks]
scores = reranker.predict(pairs)
ranked_chunks = [c for _, c in sorted(zip(scores, candidate_chunks), reverse=True)][:5]
```

Interview Answer: *"I implemented two-stage retrieval using a Cross-Encoder to compute full query-passage cross-attention and re-rank the top 5 most relevant passages."*

6. LLM Generation & Citation Verification

</> Python — LLM Inference & Citation Verification

```
import openai
client = openai.OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')
res = client.chat.completions.create(
model='deepseek-r1:1.5b',
messages=[{'role': 'system', 'content': SYSTEM_PROMPT}, {'role': 'user', 'content': prompt}]
)
clean_answer = format_and_verify_answer(res.choices[0].message.content, verified_chunks)
```

Interview Answer: *"I sent the re-ranked passages to a local Ollama LLM (deepseek-r1:1.5b), stripped reasoning traces (<think>), and programmatically verified source-page citations."*